RETAILMART V3 · SQL

AI + SQL -- Use AI as Your Tutor, Not Your Typist

— Siraj Sir

📱 WhatsApp Announcement

Bonus Session: AI + SQL -- Use AI as Your Tutor, Not Your Typist

Everyone is using AI with SQL. Most people are using it wrong -- and it shows up in interviews when they cannot debug a JOIN without a chatbot. In this bonus session we set up a FREE AI tool the professional way: it reviews YOUR queries, debugs YOUR errors, and interviews you -- it never writes your SQL for you.

Part 1: The rules -- the 80-20 way professionals use AI
Part 2: Live demos -- AI as debugger, reviewer, interviewer, and a Students-vs-AI challenge
Part 3: Your setup -- free tool install + the prompt cheatsheet

Bring: your laptop with the course repo cloned (git pull first!) and a Google account. Everything we set up is free.

TODAY'S AGENDA (55 Minutes)

Section	Time	Topics
Part 1: The Rules	15 min	Why AI dependency kills careers, The 80-20 rule, Bad usage vs good usage, Why AI needs your schema
Part 2: Live Demos	25 min	AI as debugger (hints only), AI as senior reviewer, AI as interviewer, Challenge: Students vs AI
Part 3: Your Setup	15 min	Free tool: Antigravity, The repo is already AI-ready, The daily workflow, Prompt cheatsheet in your repo

STORY

The Candidate Who Could Not Debug a JOIN

A candidate walks into a SQL interview with two years of 'experience'. The interviewer shows a five-line query with a broken JOIN and asks what is wrong. Silence. For two years, every query they 'wrote' came out of a chatbot -- they never learned to read SQL, only to request it. They became a prompt engineer who cannot debug a JOIN. Companies now test for exactly this. Today you learn the workflow that makes AI your unfair advantage instead of your replacement.

DEFINITION

80 percent you, 20 percent AI

- YOU: understand the problem, design the logic, write the SQL, predict the output
- AI: explain errors, review your query, give hints, generate fresh practice
- The 80 percent is where SQL thinking grows -- outsource it and it never grows
- The 20 percent is where AI genuinely beats books: instant, patient, personal

DEFINITION

First think. Then write. Then ask AI to review.

- THINK: what tables, what joins, what grouping -- in plain words first
- WRITE: your SQL, yourself, in psql or VS Code
- PREDICT: say out loud what the output should be BEFORE running
- REVIEW: only now does AI enter -- to react to YOUR work

COMMON MISTAKE

Bad AI usage -- the dependency trap

- 'Write SQL for this question' -- you learn nothing, and it shows in interviews
- 'Solve my assignment' -- a grade you did not earn, a skill you do not have
- 'Fix my query' without trying first -- debugging is the highest-paid SQL skill
- Copy-pasting AI output you do not understand into anything, ever

REVEAL

Good AI usage -- the professional workflow

- 'Here is my query and what I expected -- why is the output different?'
- 'Give me ONE hint, not the answer'
- 'Rate my query: correctness, performance, readability, alternative'
- 'Ask me interview questions one by one, do not reveal answers early'

STORY

The Confident Wrong Answer

Ask a chatbot for a RetailMart query without showing it the schema and it will answer instantly, confidently -- and wrong. It guesses column names that sound right. Watch what it invents:

What AI writes WITHOUT the schema (broken -- do not run)

```
-- Every line below LOOKS reasonable. Every line is wrong.
SELECT c.full_name,            -- no such column
       o.total_amount          -- no such column
FROM customers c               -- schema missing
JOIN orders o
  ON c.customer_id = o.customer_id   -- orders uses cust_id
WHERE o.created_at > '2025-01-01';   -- it is order_date
```

REVEAL

The real RetailMart columns

- customers.customers has first_name and last_name -- no full_name
- sales.orders joins on cust_id, not customer_id
- The date column is order_date -- there is no created_at
- Lesson: an AI without YOUR schema is a confident liar. Context is everything.

DEFINITION

AGENTS.md -- the tutor rules built into your clone

- The repo root now contains AGENTS.md -- run git pull to get it
- Every AI coding tool (Antigravity, Claude Code, Codex, Cursor) reads it automatically
- It tells the AI: you are a TUTOR -- hints only, never full solutions, never run SQL
- It tells the AI where the real schema lives: datasets/v3/sql/setup_accio_retailmart.sql
- So: open your cloned folder in the tool and it starts in tutor mode -- zero setup

PRO TIP

PRO TIP

Using a browser chat instead (ChatGPT / Gemini / Claude on the web)? Then YOU provide the context: open datasets/v3/sql/setup_accio_retailmart.sql from your clone, copy the whole file, and paste it as the first message with: 'This is my database schema. You are my SQL tutor. Never write complete queries for me -- hints and reviews only.'

STORY

The Most Useful Prompt You Will Ever Learn

A query fails. The old way: stare at it, give up, ask a friend. The dependency way: paste it into AI and copy the fix without understanding. The professional way: make the AI coach you to the fix. Watch the difference live.

The broken query (what is wrong here?)

```
-- Goal: orders bigger than the average order
SELECT order_id, cust_id, net_total
FROM sales.orders
WHERE net_total > AVG(net_total);

-- psql says:
-- ERROR: aggregate functions are not allowed in WHERE
```

The debugger prompt (from the cheatsheet)

```
Here is my query and what I expected it to return.

Expected: all orders with net_total above the average.

Query: <the query above>
It gives: ERROR: aggregate functions are not allowed in WHERE

Do NOT give me the corrected query. Tell me which clause the
problem is in and give me ONE hint. I will fix it myself.
```

REVEAL

What a good tutor-AI answers

Something like: 'The problem is in your WHERE clause. WHERE filters rows ONE AT A TIME, before any grouping -- but AVG(net_total) needs to look at ALL rows to compute one number. So WHERE cannot evaluate it. Hint: you need the average computed in its own little query first. Try again.' -- And now YOU write the fix.

The fix YOU write after the hint

```
SELECT order_id, cust_id, net_total
FROM sales.orders
WHERE net_total > (SELECT AVG(net_total) FROM sales.orders);
```

STORY

Code Review, On Demand

In a real job, a senior engineer reviews your SQL before it ships. Most learners never get that luxury. With the right prompt, AI becomes that senior -- and reviewing a WORKING query is where the deepest learning hides: edge cases, NULLs, performance, style.

A working query to review (run it yourself first)

```
SELECT dr.region_name,
       COUNT(*)          AS delivered_orders,
       SUM(o.net_total)  AS revenue
FROM sales.orders o
JOIN stores.stores s    ON o.store_id  = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
GROUP BY dr.region_name
ORDER BY revenue DESC;
```

The reviewer prompt (from the cheatsheet)

```
This query works. Review it like a senior engineer:

<paste the query>

Rate it on:
- Correctness (edge cases I missed? NULLs? duplicates?)
- Performance (would this survive 10 million rows?)
- Readability (naming, formatting, structure)
- Alternative (is there a cleaner approach? name it, do not write it)
```

The interviewer prompt (students love this one)

```
You are a SQL interviewer. Ask me questions one at a time
about JOINs and GROUP BY. Wait for my answer before revealing
anything. Tell me honestly if my answer is wrong and why.
Increase difficulty gradually. Start now.
```

REAL WORLD

Why This One Matters Most

This is unlimited, free, judgment-free interview practice -- available at 2 AM before your actual interview. It is the single highest-value AI habit for placement season. The AI asks, you answer out loud, it corrects you, difficulty climbs. Twenty minutes a day of this compounds fast.

CHALLENGE

Beat the AI

PROBLEM: For each region, find the single customer with the highest total delivered revenue. Show region name, customer name, and their revenue.

Round 1: YOU solve it (5 minutes, no AI).
Round 2: The AI solves it on the projector.
Round 3: Compare -- which is more readable? Which handles ties? Did the AI use a feature we have not learned? Did it get the column names right?

Answer

```
SELECT DISTINCT ON (dr.region_name)
       dr.region_name,
       c.first_name || ' ' || c.last_name AS customer_name,
       SUM(o.net_total)                   AS revenue
FROM sales.orders o
JOIN customers.customers c ON o.cust_id  = c.customer_id
JOIN stores.stores s       ON o.store_id = s.store_id
JOIN core.dim_region dr    ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
GROUP BY dr.region_name, c.customer_id, c.first_name, c.last_name
ORDER BY dr.region_name, SUM(o.net_total) DESC;
```

REVEAL

The Point of the Challenge

- AI is fast -- but was it RIGHT? You can only judge if you know SQL
- AI often over-engineers: window functions where DISTINCT ON is cleaner, or vice versa
- AI without schema context invents columns -- you saw it happen
- The skill companies pay for is not writing SQL OR prompting -- it is JUDGING output

Generate fresh practice (never solves the repo questions)

```
Generate 10 NEW practice questions on JOINs and GROUP BY using
the RetailMart schema. Difficulty: beginner. Real table and
column names only -- check the schema. Questions only, NO
answers. I will paste my attempts one by one for review.
```

AI as a fake database (practice anywhere, even on your phone)

```
Pretend you are PostgreSQL with the RetailMart schema loaded,
with a few realistic rows per table. I will type SQL. Reply
with ONLY the result table or the exact error -- nothing else.
```

Check my logic BEFORE writing SQL

```
Question I am solving: <paste question>

My plan in plain English: join orders to customers, filter to
Delivered, group by tier, keep tiers above the overall average.

Is my LOGIC correct? Do not write any SQL. Point out flaws in
the plan only.
```

DEFINITION

One free recommendation, three honest alternatives

- ANTIGRAVITY (recommended): free with any Google account, includes Gemini -- this is what we demo
- Claude Code: excellent, but real use needs a paid plan (~USD 20/month)
- OpenAI Codex: needs a paid ChatGPT plan -- skip unless you already subscribe
- Cursor: free tier is a small trial; daily use needs Pro (~USD 20/month)
- No install? Browser ChatGPT/Gemini/Claude + paste the schema file -- always works

Setup -- about 5 minutes

1. Download from https://antigravity.google
   (Windows / Mac / Linux)
2. Install, sign in with your Gmail account
3. File -> Open Folder -> your cloned postgresql repo
4. In the chat panel ask: 'What are the tutor rules here?'
   If it mentions hints-not-solutions, AGENTS.md loaded.

PRO TIP

PRO TIP

Use the CHAT side panel, not the autonomous agent mode. Agent mode is built to write and run code by itself -- that is the dependency trap with a nicer interface. You are here to think WITH a tutor. Also: the free quota resets every few hours; if you hit the limit, perfect -- go run your queries yourself for a while.

DEFINITION

How AI fits into your practice from today

- 1. Read the practice question. Think. Write your plan in plain English.
- 2. Write the SQL yourself in psql / pgAdmin / VS Code. Predict the output.
- 3. Run it YOURSELF. Wrong or broken? Try to fix it yourself first.
- 4. Still stuck after a real attempt? NOW ask AI -- for a hint, not a solution.
- 5. Query works? Ask AI for a senior review. This is where you level up.

COMMON MISTAKE

Never Do These -- With Any AI Tool

- Never give an AI tool your database password
- Never let an AI agent connect to or run queries on your database
- Never paste repo practice questions asking for solutions -- tutor mode exists for a reason
- Never submit AI output you cannot explain line by line

INTERVIEW

Q: Why do companies test SQL without AI in interviews?

A: Because on the job, AI writes the first draft but a HUMAN owns the result. When the query silently returns wrong numbers to the CFO, 'the AI wrote it' is not an answer. Companies want engineers who use AI to move faster on skills they already have -- AI-assisted, not AI-dependent. Your interview is them checking which one you are.

KEY TAKEAWAYS

KEY TAKEAWAYS

First think, then write, then ask AI to review: The 80 percent -- understanding, logic, writing, predicting -- stays yours. AI gets the 20: review, debug, hint, quiz.

AI without your schema is a confident liar: It invents column names that sound right. In a tool, AGENTS.md points it at the real schema; in a browser, paste the schema file first.

Your repo is AI-ready -- git pull: AGENTS.md puts every AI coding tool into tutor mode automatically: hints only, no solutions, no running SQL.

The five prompts that matter: Debug-with-hints, senior review, interviewer, fresh practice, logic check -- all copy-paste ready in the ai/ folder of your repo.

Judgment is the paid skill: Companies do not pay for writing SQL or for prompting -- they pay for knowing whether the output is RIGHT. That is what the 80 percent builds.

Keep Practicing

Tonight: git pull, install Antigravity, open your repo folder, and do one full loop -- pick a practice question, write your answer yourself, run it, then ask for a senior review using the cheatsheet in ai/prompts_cheatsheet.md. Then try the interviewer prompt for ten minutes. If you do that loop three times a week, you will walk into interviews with both skills they are checking for: SQL that is yours, and an AI workflow that is professional.